Name: Jyotirmoy Dutta Class: TE TI3 AI&DS ST

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

SEPM - CASE STUDY

CASE-STUDY: E-Commerce Solutions 'Transformation from Monolith to DevOps Agility

1. Problem Statement

Company Background:

E-com solutions is a successful online retailer established ten years ago. They built their entire platform as a single, large, monolithic application on their private server infrastructure.

The reasons

→ Infrequent releases: Major updates were deployed only four times a year. Each release was a massive event that took weeks to plan and often caused 12-24 hours of downtime.

→ The "Throw it over the wall" mentality.
- Development (Dev) team wrote code, compiled it, and sent a massive 200-page release document to the operations (ops) team.

→ operations (ops) team, who didn't write the code, was responsible for deploying and maintaining it. They were overwhelmed by the document and feared

deploying because they didn't understand the changes.

→ QA was a separate stage at the end, leading to bugs discovered too late.

→ High failure rate: 50% of production deployments failed or even required immediate rollback, causing major revenue loss during critical shopping periods like Black Friday.

→ Scaling issues: They could only scale the entire application. If only the checkout system was slow, they had to provision identical, massive servers for the whole stack, making scaling inefficient and expensive.

The resulting relationship was characterized by blame. Devs blamed Ops for being slow; Ops blamed Devs for poor quality. This is the classic wall of confusion that DevOps is designed to tear down.

2. Solution: Adopting DevOps Principles

E-Com Solutions realized they couldn't just buy new tools; they needed to change their culture and adopt the C.A.L.M.S framework of DevOps:

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Culture: They must stop blaming and start collaborating. Dev and ops are one delivery team with shared goals (stability and velocity)

Automation: They must automate repetitive, manual tasks to eliminate human error and speed up the process. This is the heart of the new pipeline.

Lean: Process bottlenecks and waste (like waiting weeks for server provisioning or massive release documents) must be identified and removed.

Measurement: They must shift from guessing to data-driven decision-making, metricizing key performance indicators (KPIs)

Sharing: Knowledge, tools and processes must be open and accessible across the team

3. Practices Implemented in the Transformation

To the realize these principles, E-commerce solutions adopted several key technical practices.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Devops practice	How E-comsolutions implemented it	Value realized.
Continuous Integration (CI)	Every developer's code change is committed daily to a central respo repository (Git). Each commit triggers an automatic build and unit test.	Bugs are caught instantly in small code batches, not months later on a massive merge.
continuous delivery (CD)	The validated build is automatically deployed to a production-like staging environment for further automated testing. The artifact is always ready for production	High confidence in release quality. releasing becomes boring and non-eventful.
Infrastructure as code	The team started using tools like Terraform to define their cloud infrastructure using simple text files, stored in Git	Environments are now identical. No more "it worked on my machine" issues. Scaling up infrastructure takes minute not weeks.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

Automated testing	QA engineers started writing automated test suites that run within the CI/CD pipeline (unit, integration, regression).	manual regression testing (days) is replaced by automated tests (minutes)
micro services Architecture	They began breaking the large monolith into small, loosely coupled services (eg:- inventory service, payment service, user service).	Services can be developed, tested, deployed and scaled independently, making the system more resilient and faster to iterate.
monitoring and logging	Implemented robust distributed tracing (Prometheus, Grafana) and centralized logging (ELK stack) to monitor application health in real-time.	Issues are detected immediately, often before the customer notices. MTTD (mean time to detect) drops.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

4. The Devops Engineer role and responsibilities

During this transformation, E-com solutions realized they needed a new specialized role to facilitate the change. They didn't just needed operators or sysadmins, they needed Devops Engineers

A Devops Engineer is a hybrid professional who understands both coding/scripting and systems infrastructure. They Their primary goal/Job is to engineer the delivery pipeline.

Key Devops Engineer responsibilities (mapped to E-com solutions journey):

1. Orchestrating the Toolchain: The Devops Engineer is responsible for integrating Jenkins, Git, Terraform, and kubernetes into a single, cohesive CI/CD pipeline.

2. Infrastructure as code (IaC) Authoring: They write the ~~Terraform~~ Terraform code to define the environments. Instead of manually clicking in the Aws console, they commit code that automatically provisions the infrastructure.

3. 3. Pipeline Automation & maintenance: They write the scripts (eg:- Python, Bash) that make the pipeline run - automating code testing, container image building, and deployment orchestration.

THADOMAL SHAHANI TSEC ENGINEERING COLLEGE

→ Cloud architecture & containerization: They manage the migration from vms to containerized application environments (like kubernetes / EKS), optimizing resource usage and net reliability.

→ monitoring and incident response: They establish the monitoring and logging stack (ELK / prometheus). define alerting rules, and actively participate in incidents, ensuring they have the telemetry telemetry to diagnose problems quickly.

→ Security integration (DevSecOps): (An emerging need in E-com's journey). The DevOps engineer integrates automated security scanning (SAST / DAST) directly into the CI/CD pipeline.

E-Com solutions: Before vs. after the DevOps Transformation

→ After 18 months of cultural change and automation (the journey shown in images 0 and 1), the results were dramatic:

metric	Before DevOps	After DevOps
Deployment Frequency	4x per year	multiple times per day
lead time for changes	weeks	minutes / hours
Change failure rate (prod)	~50%	< 5%
mean time to recovery	Days / weeks	minutes
Customer satisfaction	low	high